

cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith^{1,*}, Gregory Koytiger^{1,*}

¹Synthesize Bio

*Correspondence:

maxrhighsmith@gmail.com; greg@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression analysis. Its gene-wise dispersion, generalized linear-model, and effect-size-shrinkage calculations expose substantial parallelism, yet their iterative and numerically sensitive structure has limited faithful GPU implementations.

Results: We present cuDESeq2, a GPU implementation that preserves the DESeq2 statistical workflow while batching gene-wise computation. Comparing eager mode with R DESeq2 1.52.0 across six real designs (7–300 samples, $P = 2-5$), final dispersion has a median p95 relative error of $8.5e-4$, significant-gene sets have Jaccard similarity of at least 0.99963, and shrunk LFCs have Pearson correlation of at least 0.99722. On one A100, the fastest cuDESeq2 mode is $3.6-35.1\times$ faster than a practical 12-worker R baseline in direct end-to-end measurements. A separate real-GTEX scaling study runs 54,922 genes across 2,451 samples and a $P = 6$ tissue design in a 22.936-s direct median, $28.0\times$ faster than the 12-worker R endpoint.

Availability and implementation: Open source under the MIT license at https://github.com/synthesizebio/gpu_deseq; PyTorch and a CUDA GPU are required for the reported acceleration.

Contact: maxrhighsmith@gmail.com; greg@synthesize.bio

1 Introduction

Differential-expression analysis is a central step in interpreting RNA-seq experiments. DESeq2 (Love et al., 2014) is widely used because it combines a negative-binomial model of count variability with empirical-Bayes dispersion estimation, outlier handling, multiple-testing correction, and stable effect-size estimation. These features make it more than a gene-wise hypothesis test: the pipeline shares information across genes while retaining gene-specific means, dispersions, coefficients, and diagnostics.

This statistical structure also creates a distinctive computational problem. For tens of thousands of genes, dispersion estimation, generalized linear-model fitting, and log fold-change (LFC) shrinkage (Zhu et al., 2019) repeatedly solve related but independent optimization problems. R DESeq2 executes this work on CPUs and distributes blocks of genes among workers. The independence across genes suggests GPU parallelism, but a faithful implementation cannot be obtained by replacing a single matrix operation. Per-gene convergence, line searches, small design-matrix solves, empirical-Bayes steps, and exceptional-gene fallbacks must all preserve the reference method’s statistical decisions (Fig. 1).

Wall time is often acceptable when differential expression is performed only a few times in an individual study. It becomes limiting when the analysis is repeated across many cell states, perturbations, contrasts, or model outputs. The bioinformatics community has recently seen growing interest in virtual-cell systems and foundation models for gene-expression data (Theodoris et al., 2023; Cui et al., 2024; Bunne et al., 2024; Adduri et al., 2025; Koytiger et al., 2025). Differential expression is a natural evaluation endpoint for such models, yet some recent benchmarks use rank-based tests on normalized expression as less expensive surrogates (Roohani et al., 2025; Wei et al., 2026). Accelerating the established count model would allow statistical rigor and computational throughput to be improved together rather

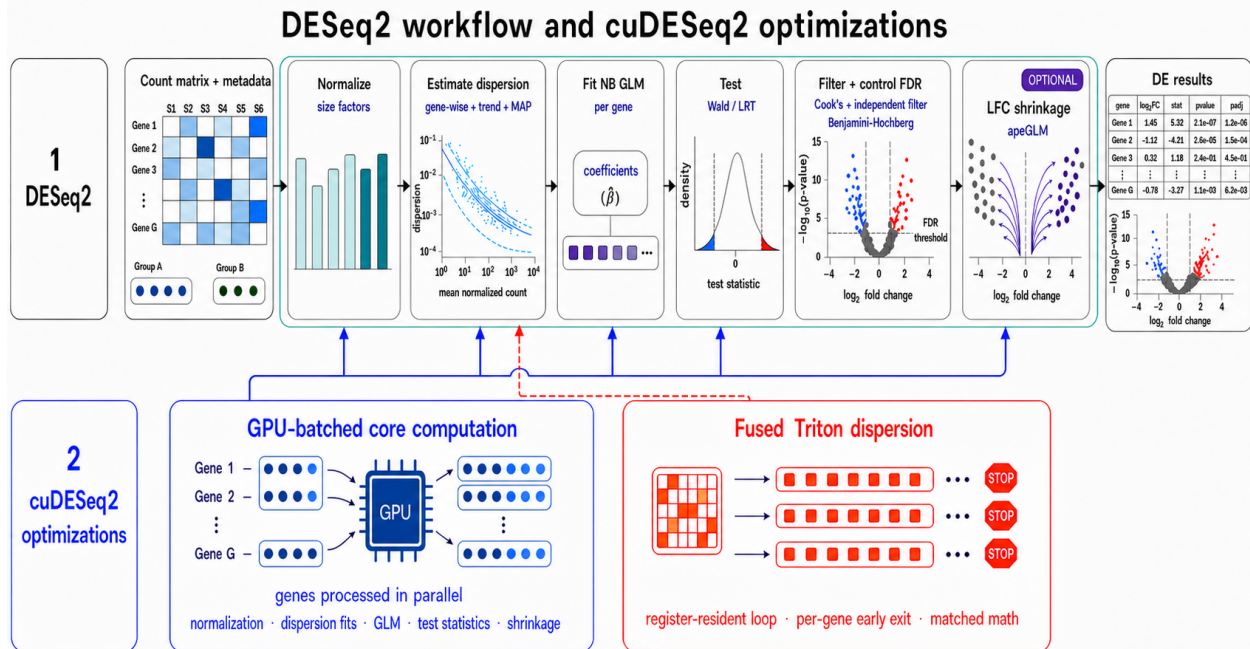


Figure 1: Overview of cuDESeq2. *Top*, the five stages of a DESeq2 run. *Bottom*, where the GPU work goes: DESeq2’s per-gene, sequential CPU work (dispersion estimation, the negative-binomial GLM, and apeGLM LFC shrinkage) is batched across all genes (*left*, solid arrows), and the dispersion fit, which is limited mainly by launch latency, has a fully fused Triton implementation (*right*, dashed arrow). A third mode uses CUDA-graph chunked replay; the three modes are compared in Table 1. Every stage is checked against R DESeq2 step by step (Sec. 3.1).

than traded against one another.

Existing efforts to accelerate negative-binomial differential expression have largely been algorithmic or CPU-bound. glmGamPoi (Ahlmann-Eltze and Huber, 2020) accelerates Gamma–Poisson GLM fitting on large, sparse count matrices. However, replacing DESeq2 with glmGamPoi changes the inferential workflow and forgoes DESeq2’s automatic outlier replacement, limiting comparability with established DESeq2 results and removing a safeguard against influential counts. PyDESeq2 (Muzellec et al., 2023), a Python reimplementation, improves interoperability and speed on large datasets, though by its authors’ own account it yields results “similar, but not identical” to DESeq2; GPU acceleration has meanwhile transformed adjacent single-cell workflows, where rapids-singlecell and ScaleSC report 15–20× end-to-end speedups (Hu et al., 2025), but those gains come from the dense and sparse

linear-algebra core: PCA, neighbor graphs, and clustering. Whether an iterative, empirically regularized workflow such as DESeq2 can achieve comparable acceleration without materially altering its outputs has therefore remained unresolved.

Here we test the hypothesis that DESeq2’s gene axis can be treated as a batch dimension while retaining its model, optimizer decisions, and downstream inference. We introduce cuDESeq2, which implements the complete supported workflow in PyTorch (Paszke et al., 2019), including count-outlier replacement and apeGLM shrinkage. In addition to a general batched implementation, we develop CUDA-graph (NVIDIA Corporation, 2024) and Triton (Tillet et al., 2019) dispersion paths that address launch overhead and irregular convergence. We evaluate both numerical agreement and end-to-end performance on published RNA-seq datasets, then use nested GTEx cohorts to characterize scaling with sample count, design width, and GPU memory.

2 Methods

2.1 Reference workflow and supported scope

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM. For gene g with counts K_{gi} over samples $i = 1, \dots, S$, design matrix $X \in \mathbb{R}^{S \times P}$, and size factors s_i , it assumes $K_{gi} \sim \text{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$. Size-factor normalization accounts for library depth; gene-wise dispersions are moderated toward a mean–dispersion trend; negative-binomial GLMs estimate condition effects; Wald or likelihood-ratio tests quantify evidence against the null; and independent filtering and multiple-testing correction produce adjusted significance estimates. Optional apeGLM shrinkage regularizes noisy LFCs while leaving the hypothesis-test p-values unchanged. cuDESeq2 preserves this sequence and the information passed between stages.

2.1.1 Normalization

Median-of-ratios normalization is implemented with GPU tensor reductions and quantiles.

80 The default `sfType="ratio"` raises, as DESeq2 does, when every gene contains a zero. DESeq2's alternative `sfType="poscounts"` estimator is also implemented for such sparse matrices.

2.1.2 Dispersion estimation

Gene-wise dispersion fitting for both the MLE and MAP estimators is batched across genes.

85 The optimization methods used for this stage are described in Sec. 2.2.

2.1.3 GLM fitting and hypothesis testing

Negative-binomial IRLS fits and Wald or likelihood-ratio tests are batched over genes; the implementation details are given in Sec. 2.3.

2.1.4 Significance filtering and result assembly

90 Cook's filtering, independent filtering, and multiple-testing adjustment are implemented as vectorized array operations. For CUDA analyses, the robust Cook's-distance calculation, thresholding, and count replacement operate on the resident GPU matrices. Independent filtering obtains the rejection counts for its 50 nested thresholds from one p-value ordering and performs the full Benjamini–Hochberg adjustment only at the selected threshold. The
95 standard wrapper performs replacement and refitting when DESeq2's eligibility conditions are met.

2.1.5 LFC shrinkage

The apeGLM optimizer is batched across genes on the GPU; Sec. 2.4 describes the implementation.

2.2 Dispersion acceleration

The dispersion stage is a batched port of DESeq2’s `fitDisp` and the supporting `estimateDispersions` logic. Its gene-wise MLE maximizes the Cox–Reid adjusted profile log-likelihood

$$\ell_{\text{CR}}(\alpha_g) = \sum_{i=1}^S \log \text{NB}(K_{gi}; \mu_{gi}, \alpha_g) - \frac{1}{2} \log \det(X^\top W_g X), \quad (1)$$

where $W_g = \text{diag}(\mu_{gi}/(1 + \alpha_g \mu_{gi}))$. We retain DESeq2’s initialization, line search, trend fit, MAP shrinkage, and outlier rule. Because dispersion is a significant source of pipeline compute time, we expand optimization beyond basic batching and provide three alternative implementations of the gradient-ascent loop: eager, CUDA-graph, and Triton. The three implementations use the same optimizer decisions and differ only in how those evaluations are scheduled.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \dots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P = 1$ or $P > 6$. Triton’s agreement with eager is quantified in Sec. 3.1.

CUDA-graph chunked replay. In eager mode each gradient-ascent iteration launches a handful of tiny kernels over 1–10 MB arrays, so the loop is dominated by launch latency. Capturing the loop as a CUDA graph amortizes that cost into a single replay, but two obstacles must be overcome.

The first is early exit. A fixed full-length capture would force every gene to the maximum iteration count, which wastes the typical 15–40-iteration case. To prevent this we capture a $K=10$ -iteration graph that reads and writes a set of persistent state buffers in place, and replay it in a loop with a convergence check between chunks. A chunk always runs to its

boundary, so a gene that converges partway through executes further iterations. We selected ten iterations per replay empirically to balance replay overhead against excess iterations
 120 after convergence.

The second is cuSOLVER. The Cox–Reid term needs $\log \det(b)$ and $\text{tr}(b^{-1}db)$ with $b = X^\top W X$, but `slogdet/solve` host-synchronize and cannot be captured. Since b is symmetric positive-definite for full-rank X , we replace them with an unrolled no-pivot LU (`_logdet_and_tr_binv_db`) returning both quantities from elementwise and indexing
 125 operations alone, matching the `linalg` path to $\sim 1\text{e-}15$ for $P = 2 \dots 6$.

Before capture the chunk is run three times on a side stream, warming the caching allocator and any lazy kernel initialization. Captured graphs are cached on G , S , P , the chunk length K , whether a prior term is enabled, the dtype and the device. Only *whether* there is a prior is selected on the device, because the prior variance lives in the buffer set as
 130 a scalar tensor the replay reads, so one graph serves datasets whose prior variances differ. Because the graph uses the same routine, it retains bit-identical parity with the eager mode implementation.

Triton full-loop fusion. Each Triton dispersion fit uses one kernel launch over a grid with one program instance per gene (Fig. 2). Each instance loads its counts, μ , and the design
 135 columns once into registers, then runs the iterative gradient-ascent loop register-resident: the Cox–Reid $P \times P$ solve (closed-form 2×2 for $P=2$, an unrolled no-pivot LU above that), the NB log-likelihood and $\log \alpha$ -gradient (`lgamma` and `log1p` from `libdevice`, `digamma` hand-implemented to match ATen’s `calc_digamma` through a recurrence to $x \geq 10$ followed by the Bernoulli asymptotic series), and the Armijo line search. A per-gene `while` loop lets
 140 each gene exit as soon as it converges, so the kernel does not run every gene to the batch maximum. The kernel is fp64 and covers $P \in \{2, \dots, 6\}$; `fit_dispersions` gates on the covered range and routes wider designs to the eager path, while the kernel’s own launcher raises instead of fitting a truncated model, since it reads a fixed number of design columns.

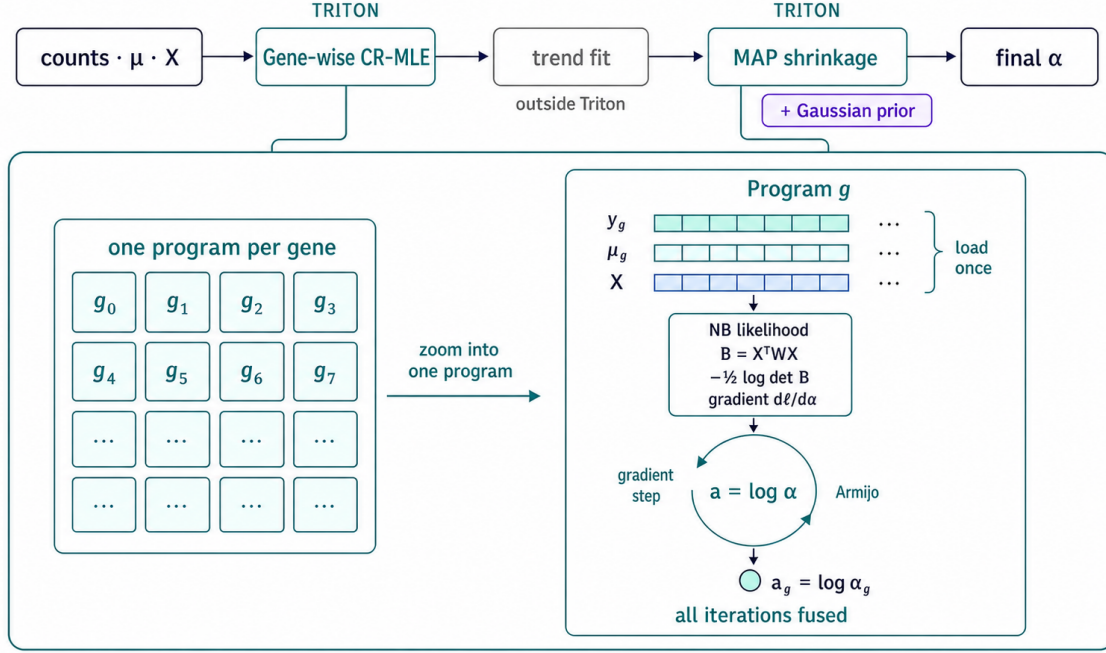


Figure 2: The fused Triton dispersion kernel. The same kernel implementation is invoked separately for the two gene-wise fits of the dispersion pipeline: the Cox–Reid MLE and the MAP shrinkage with a Gaussian prior that follows the trend fit (the trend fit itself runs outside Triton). Each invocation is one kernel launch over a grid with one Triton program instance per gene. Each instance loads $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and the design $\mathbf{X}$ once, then runs every iteration of the gradient ascent register-resident, with the NB log-likelihood, the Cox–Reid term $-\frac{1}{2} \log \det \mathbf{B}$ ($\mathbf{B} = \mathbf{X}^\top \mathbf{W} \mathbf{X}$), the $\log \alpha$ -gradient and the Armijo line search all fused into the loop body. Only the converged $a_g = \log \alpha_g$ and a diagnostic iteration count are written back to global memory. Supplementary Algorithms S1–S2 give the pseudocode.

At the gene-wise step, non-converged and high-dispersion genes get the same noIncrease
 145 revert and grid fallback as the eager path, computed eagerly after the kernel returns; at the
 MAP step neither applies, in either mode, because R applies neither. The fused reduction
 order and the hand-written `digamma` move the last bits, so this mode is not bit-identical to
 eager, and Sec. 3.1 reports how far the two diverge on real data.

2.3 GLM acceleration

150 The GLM is implemented as a batched negative-binomial IRLS solve on the GPU, using
 one batched factorization across genes at each iteration. Genes that diverge or reach the
 DESeq2 iteration cap use the same host-side fallback as the reference; only those exceptional
 gene rows are transferred. Cook’s robust dispersion, distance, count replacement, and the
 affected-gene refit remain on the GPU in CUDA workflows. Wald standard errors retain the
 155 $\mu \geq 0.5$ floor in `fitBeta.cpp`; LRTs use the full–reduced deviance difference.

2.4 LFC shrinkage acceleration

apeGLM (Zhu et al., 2019) is a per-gene optimization, so its L-BFGS state, line search, and
 convergence flags are batched on the GPU. We match the reference optimizer’s initialization
 and stopping rules because extreme, low-count genes can have competing posterior modes.
 160 As in `lfcShrink`, only the shrunken coefficient and its standard error are replaced; p-
 values remain those from the Wald fit. Loss and gradient share the same large gene–sample
 linear predictor, invariant count/dispersion terms are hoisted out of the optimizer loop, and
 fitted size factors and Wald inference are reused rather than reconstructed downstream. On
 CUDA, PyTorch compilation fuses the elementwise likelihood, score, and final-Hessian work
 165 surrounding the matrix products. This removes repeated gene–sample temporaries while
 retaining the same L-BFGS initialization, line search, stopping rules, and fp64 outputs;
 CPU execution keeps the eager implementation.

2.5 Validation and benchmark design

Reference fixtures were generated with R DESeq2 1.52.0: three single-factor designs at 12×200 , 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim$ batch + condition with a three-level batch ($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$), using the seeds 0, 0, 1, 2, and 3, respectively. Every intermediate is checked against R: size factors, `dispGeneEst`, `dispFit` (the parametric trend on every fixture, plus the "local" and "mean" trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β , μ , the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the `apeGLM` prior scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.1 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2). For real-data summaries, max relative error is used for positive size factors; p95 relative error for positive dispersions; p95 absolute error for LFCs, which cross zero; Jaccard similarity of finite `padj` < 0.05 sets; and Pearson correlation over finite pairs for shrunk LFCs. Empty called-set unions are assigned Jaccard similarity one. Eager, graph, and Triton modes were evaluated separately against the same reference and acceptance thresholds.

Performance was evaluated on one NVIDIA A100-SXM4-40GB under PyTorch 2.7.1, CUDA 12.6, and NVIDIA driver 580.126.20. The CPU reference was R DESeq2 1.52.0 with `apeglm` 1.34.0 on a 12-vCPU virtual machine. The primary comparison used the ordinary `DESeq()`, `results()`, and `lfcShrink()` call sequence with `MulticoreParam(12)`. A one-worker run was used only to attribute time to comparable pipeline stages. GPU times are medians of five complete, CUDA-synchronized measurements after one warm-up. The two long-running large-GTEX CPU endpoints are single cold observations and are interpreted as endpoint comparisons rather than sampling distributions. Memory-boundary experiments are single fresh-process feasibility tests.

The study includes six designs derived from three published datasets: pasilla (Brooks

et al., 2011) ($n=7$; one- and two-factor designs), airway (Himes et al., 2014) ($n=8$; three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood versus skeletal-muscle contrast with 54,922 genes. GTEx counts were obtained from recount2’s uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017); the primary contrast contains 150 samples per tissue selected using R seed 1. Base coverage was converted to read counts using each sample’s average read length, with 100 bases used for missing or non-positive values. A fixed-seed simulation independently varies sample count at 2,000 genes. Finally, nested cohorts from all 9,662 GTEx source samples and 54 tissue levels vary sample count and design width while holding the gene axis fixed.

3 Results

3.1 GPU execution preserves DESeq2 results

We compared each stage of the pipeline with R DESeq2 1.52.0 across six real-data designs derived from three datasets (Table 2). Agreement was evaluated separately for normalization, dispersion estimation, GLM fitting, significance calls, and LFC shrinkage.

Table 2: Agreement with R DESeq2 1.52.0 across six real-data designs. Each design was evaluated independently using a metric appropriate to the indicated pipeline stage.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $ \Delta $ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

Every eager-mode pipeline stage met its predefined tolerance across all six real-data designs. Normalization was effectively identical to R, and the largest dispersion discrepancy was 0.44% at the 95th percentile. Raw LFC estimates differed by at most 8.1×10^{-5} at the 95th percentile, and significant-gene sets had Jaccard similarity of at least 0.99963. Shrunk

LFCs had Pearson correlation of at least 0.99722 with R.

3.1.1 Parity in the standard DE plots

215 In addition to summary statistics we validate against standard diagnostic plots used in DE analysis. Figure 3 displays dispersion-estimation, MA, and volcano plots from both pipelines. The gene-wise MLE cloud, the fitted trend, the shrunken MAP band, and the outlier pattern visually overlap; both implementations call 3,993 genes at $\text{padj} < 0.05$ in this design.

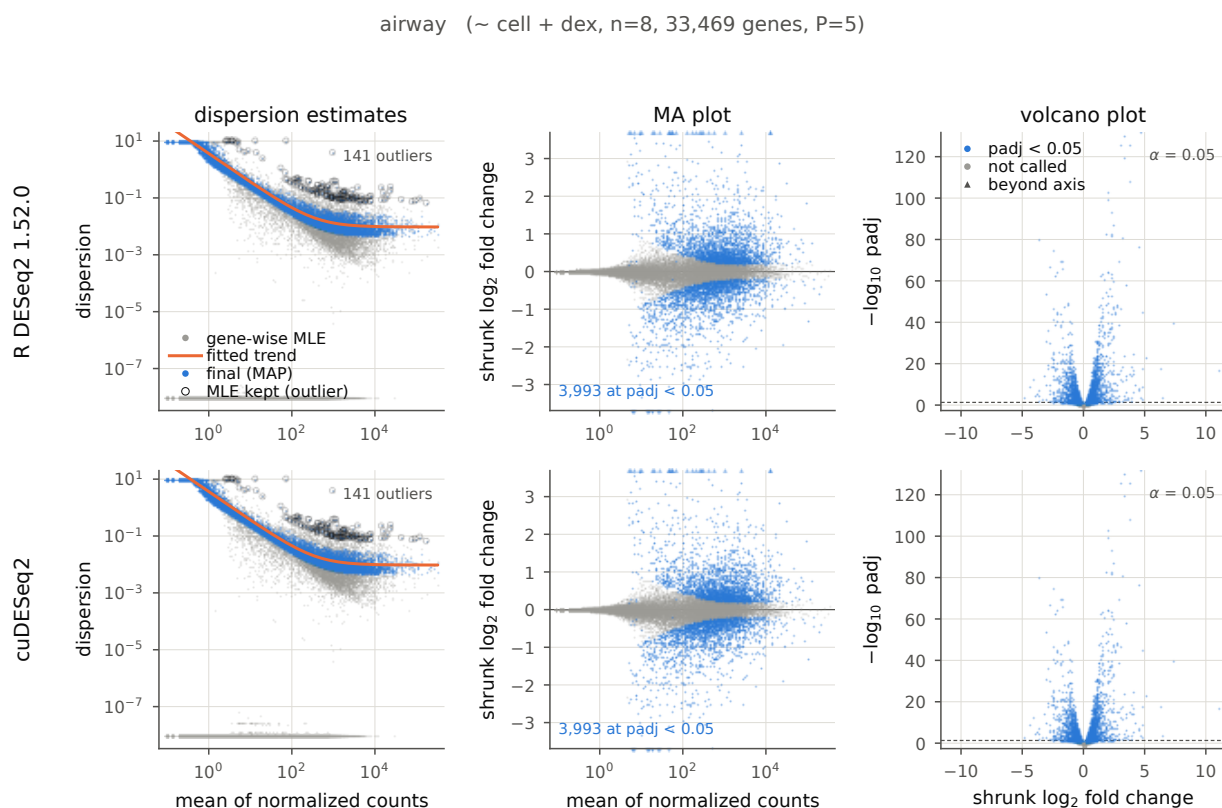


Figure 3: The three standard DESeq2 diagnostics on `airway` ($\sim \text{cell} + \text{dex}$, $n=8$, $P=5$), drawn from R DESeq2 1.52.0 (top) and cuDESeq2 (bottom) by the same plotting code. *Left*, the `plotDispEsts` view: gene-wise MLE (grey), fitted trend (orange), final estimate (blue), and circled genes whose MLE exceeds the trend by more than two residual SDs and is therefore kept unshrunk. *Center*, MA plot of the apeGLM-shrunk LFC, with out-of-range genes on the boundary as in `plotMA`. *Right*, volcano plot. Axis limits are computed from R's output and reused for cuDESeq2.

The two pipelines also produce comparable ranked genes (Fig. 4). At the prespecified

220 $\alpha=0.05$ threshold, four of the six called sets agree exactly and the other two reach 0.99963 and 0.99997.

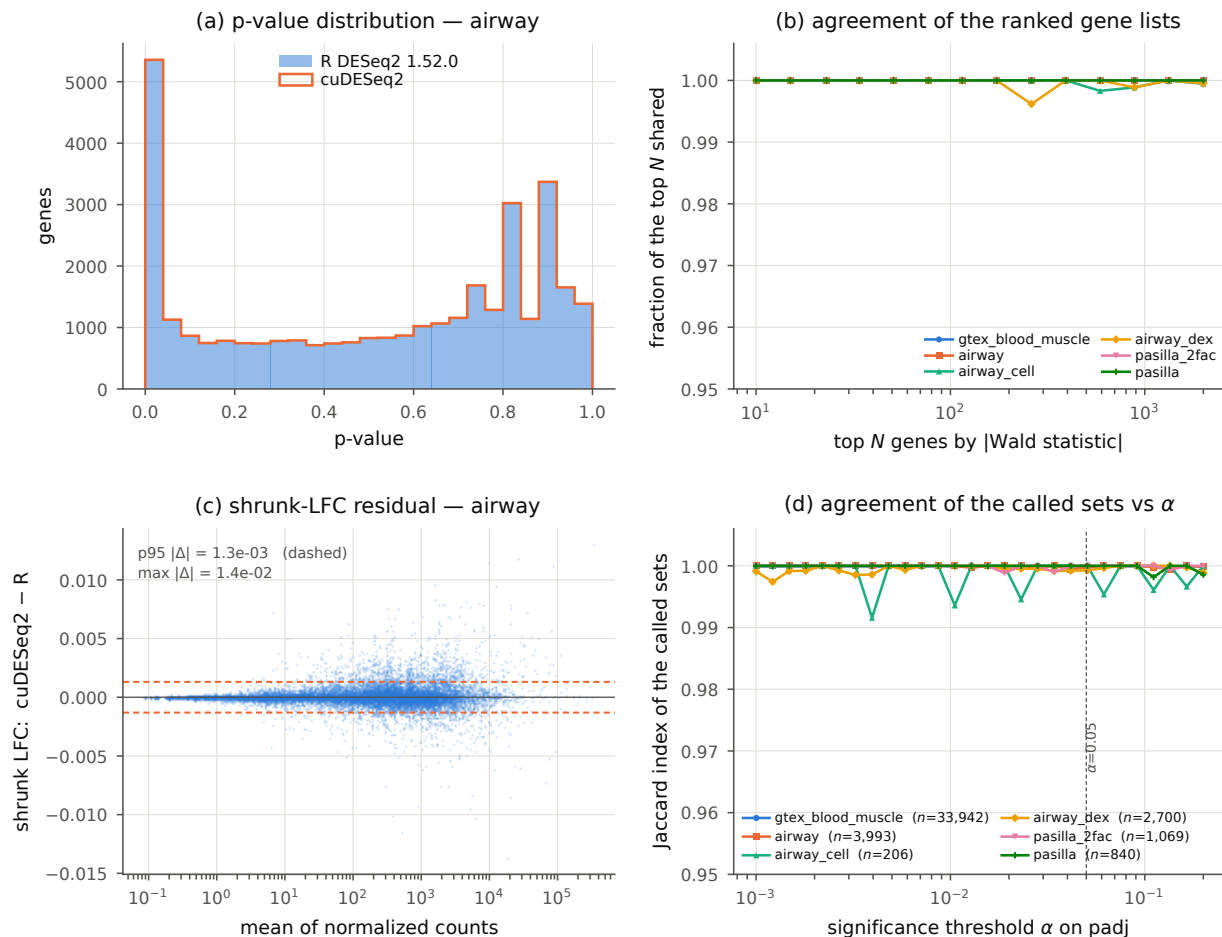


Figure 4: Quantitative agreement across all six designs. (a) p-value distributions on *airway*, overlaid. (b) Fraction of the top N genes shared by the two rankings, ranked by $|Wald\ statistic|$ to avoid comparing tie-breaks among genes whose p-value has underflowed in R. (c) apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. (d) Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at $\alpha=0.05$ in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The *gtex* curve now includes count replacement and refitting in both pipelines.

3.2 GPU acceleration reduces end-to-end wall time

Across the six real-data designs, every GPU mode reduced wall time relative to 12-worker R DESeq2, although the magnitude depended on the model and dataset (Table 3). Graph

and Triton execution alter only the scheduling of dispersion estimation relative to eager cuDESeq2. The controlled stage breakdown in Table 4 therefore isolates the source of their additional gains.

Table 3: End-to-end wall time on the real datasets. R uses 12 `BiocParallel` workers; GPU columns are the three cuDESeq2 modes. Values are medians of five complete observations and include data-object construction, inference, result assembly, LFC shrinkage, and any required count-outlier refitting.

Dataset	P	n	R 12w (s)	eager (s)	graph (s)	Triton (s)	best GPU vs. R
pasilla	2	7	7.336	0.972	0.588	0.525	14.0×
pasilla_2fac	3	7	7.371	1.541	0.932	0.848	8.7×
airway_dex	2	8	11.856	1.332	0.858	0.790	15.0×
airway_cell	4	8	10.421	3.596	3.003	2.870	3.6×
airway	5	8	12.099	2.479	1.552	1.401	8.6×
gtex	2	300	63.445	5.284	4.910	1.807	35.1×

Against the practical 12-worker CPU baseline, the best direct GPU mode is 3.6–35.1×

faster, with `airway_cell` and `gtex` as the endpoints. Triton is the fastest total mode and has

the fastest dispersion substep on all six designs: fusing the wider Cox–Reid solve (Sec. 2.2)

cuts the dispersion substep by 8.9×

on `pasilla_2fac` ($P=3$) and 2.8×

on `airway` ($P=5$)

against the eager loop (777→87ms and 1660→585ms, same run). `airway`’s dispersion

substep also includes the CPU R-stream replay of Sec. 2.2 in every mode. Although it

typically lags Triton, graph is bit-identical to eager (Sec. 3.1).

3.3 Fused dispersion remains efficient as sample count grows

To distinguish scaling in sample count from scaling in gene count, we fixed 2,000 simulated

genes and varied the balanced two-group design from 4 to 2,000 samples (Table 5). The full

dispersion stage includes gene-wise estimation, trend fitting, and MAP shrinkage. Iteration

counts vary with the simulated input, so wall time need not increase monotonically with

sample count. Graph replay is most useful while launch overhead dominates, whereas the

fused Triton path remains fastest throughout this sweep.

Table 4: Pipeline-stage wall time (ms) on the real datasets. The one-worker R measurement is a diagnostic for stage attribution rather than the practical performance baseline. `glm_fit` includes the initial Wald fit and count-outlier replacement/refitting; **stage sum** is the sum of stage medians rather than a separately observed end-to-end duration.

Dataset	Substep	R 1w diagnostic	eager	graph	Triton
pasilla	dispersion	1782	463	78	21
	glm_fit	2079	42	42	41
	significance	209	40	40	40
	lfc_shrink	5343	420	422	421
	stage sum	9573	966	583	524
pasilla_2fac	dispersion	2179	777	174	87
	glm_fit	1495	354	354	352
	significance	184	50	50	49
	lfc_shrink	6082	347	346	347
	stage sum	10104	1528	925	837
airway_dex	dispersion	4481	581	141	56
	glm_fit	5171	58	59	57
	significance	1387	82	109	82
	lfc_shrink	14843	573	578	576
	stage sum	26070	1296	888	773
airway_cell	dispersion	5456	801	198	62
	glm_fit	3875	39	38	39
	significance	332	78	78	75
	lfc_shrink	20271	2675	2667	2674
	stage sum	30122	3593	2982	2852
airway	dispersion	7863	1660	723	585
	glm_fit	4398	62	62	64
	significance	310	74	74	73
	lfc_shrink	20189	673	685	663
	stage sum	32961	2470	1545	1386
gtex	dispersion	187567	3574	3612	217
	glm_fit	53421	849	491	432
	significance	456	85	73	95
	lfc_shrink	42474	570	569	929
	stage sum	287354	5081	4749	1678

normalization omitted (≤ 4 ms cuDESeq2 on every set). All GPU values come from the same A100 session.

Table 5: Sample-count scaling of the full dispersion stage (ms) at 2,000 genes. Values are CUDA-synchronized medians of seven measurements after three warm-ups on one A100; the simulator seed is 7919*n*.

Samples	eager	graph	Triton
4	1075	636	498
6	627	161	32
30	187	68	28
60	238	61	30
200	239	113	34
1000	337	282	23
2000	474	402	90

3.4 Real GTEx cohorts reveal scaling and memory limits

The synthetic sweep isolates the sample axis but does not measure the complete workflow on a large real matrix. We therefore analyzed all 9,662 samples in the recount2 GTEx study at a fixed 54,922 genes. A nested $P = 2$ series increases sample count within whole blood and skeletal muscle, whereas a second series adds abundant tissues to increase both cohort size and design width. This construction exposes two distinct scaling regimes: the number of observations contributing to each gene-wise objective, and the size of the per-gene linear-algebra problem.

Table 6: Full-workflow real-GTEx scaling on one A100 at 54,922 genes. Direct wall time is the median of five CUDA-synchronized observations after one warm-up; peak allocation is the maximum over five staged repetitions. $P=2-6$ uses the fused Triton dispersion path.

Cohort	Samples	P	Direct (s)	Peak GiB
paper-equivalent	300	2	1.753	1.98
2 tissues, balanced	600	2	2.740	3.95
2 tissues, source max.	912	2	5.430	5.99
3 tissues $\times$ 300	900	3	6.005	5.92
4 tissues $\times$ 300	1,200	4	8.990	8.38
5 tissues $\times$ 300	1,500	5	9.898	11.09
6 tissues $\times$ 300	1,800	6	14.323	14.77
6 tissues, all samples	2,451	6	22.936	20.10

For the nested $P=2$ series, tripling the real sample count from 300 to 912 increases direct GPU time from 1.753 to 5.430 seconds and peak allocation from 1.98 to 5.99 GiB. The widest fused case uses all samples from the six largest tissues: 2,451 samples at $P = 6$

complete in a 22.936-s direct median with 20.10 GiB peak allocation. We also measured the same direct workflow with 12-worker R DESeq2 (Table 7). The GPU is $39.4\times$ faster at 912 samples/ $P = 2$ and $28.0\times$ faster at 2,451 samples/ $P = 6$. Because the complete CPU calls require 3.6–10.7 minutes, each R value is a single observation; these ratios are endpoint comparisons rather than estimates with repeated-sampling uncertainty.

Table 7: Large-GTEx direct endpoint observations against the practical 12-worker R baseline. R is one cold observation in a fresh process; GPU is the direct median of five warm observations. All columns include dataset construction and the complete workflow. “GPU vs.” reports R time divided by GPU time.

Samples	P	GPU (s)	R 12w (s)	GPU vs. R 12w
912	2	5.430	213.896	$39.4\times$
2,451	6	22.936	642.552	$28.0\times$

As a validation check, serial and 12-worker R output tables agree to floating-point precision: at $P=6$ the largest relative dispersion difference is 1.53×10^{-14} , the largest raw- or shrunk-LFC difference is 9.99×10^{-15} , and the significant sets are identical. The independent GPU–R endpoint parity checks use the same five predefined tolerances as Table 2. At $P=6$, dispersion p95 relative error is 0.00212, raw-LFC p95 absolute error is 3.18×10^{-5} , significant-set Jaccard is 0.99896, and shrunk-LFC Pearson correlation is 0.999994; all five stages pass. Dispersion agreement was evaluated at the same point in both workflows, immediately after dispersion estimation and before count-outlier refitting.

Finally, nested all-sample tissue models were extended beyond the fused kernel’s $P \leq 6$ limit, where cuDESeq2 uses the eager dispersion path (Table 8). A $P = 9$ model with 3,478 samples passes at 37.09 GiB peak allocation, while $P=10$ with 3,784 samples fails during dispersion. Thus the observed boundary on this 40-GB A100 lies between those two nested cohorts; it is not a universal sample-count limit because both sample count and design width change.

Table 8: Observed A100 feasibility boundary for nested all-sample GTEx tissue models. Each row is one cold workflow in a fresh process. $P > 6$ uses eager dispersion fallback; pass-row wall times are not treated as timing estimates.

Tissues	Samples	P	Peak GiB	Status	Failed stage
8	3,147	8	30.98	pass	—
9	3,478	9	37.09	pass	—
10	3,784	10	27.88	OOM	dispersion
12	4,338	12	35.51	OOM	dispersion

4 Discussion

The principal finding is that the full supported DESeq2 workflow can exploit GPU parallelism without substituting a different statistical test. Across six real-data designs, cuDESeq2 preserved normalization, dispersion estimation, GLM coefficients, significance calls, and apeGLM-shrunken effects while reducing observed end-to-end wall time by $3.6\text{--}35.1\times$ relative to 12-worker R DESeq2. Agreement at intermediate stages is important here: similar final gene lists alone could conceal compensating differences in dispersion, standard-error, or multiple-testing calculations.

The results identify the gene axis as the useful unit of parallelism for this class of count model. Basic batching exposes thousands of independent fits to the GPU, but dispersion estimation also illustrates why batching alone is insufficient. Its small matrix operations and variable-length optimization loops can be limited by dispatch overhead rather than arithmetic throughput. CUDA-graph replay reduces that overhead without changing eager arithmetic, whereas the Triton implementation fuses the complete per-gene loop and permits independent convergence. The relative benefit varies with design width and sample count because the balance shifts among dispersion, GLM fitting, significance processing, and LFC shrinkage. This explains why stage-level acceleration does not translate into a constant end-to-end speedup and points to shrinkage as an important target for further optimization in some designs.

A broader methodological observation is that reference compatibility depends on both the statistical specification and details of its numerical realization. The dispersion prior, Wald

standard errors, and apeGLM optimization each expose choices that are not fully determined by the model equations. For example, the minimum fitted-mean convention used in the Wald
295 weight matrix affects genes with near-zero fitted counts and can change their standard errors. Conversely, the Triton kernel evaluates the same dispersion objective as eager execution but changes floating-point reduction order and the implementation of the digamma function. Its maximum absolute eager–Triton dispersion difference is 0.2604 on GTEx and no more than 3.8×10^{-7} across the five smaller designs. The downstream agreement reported here
300 indicates that these numerical differences do not constitute a practically meaningful change to inference on the evaluated data, but it does not imply bitwise identity.

The GTEx experiments extend the conclusion beyond the small public examples. At 54,922 genes, the fused path supported a 2,451-sample, six-tissue model in less than half the memory of the 40-GB A100, while wider eager models reached the memory boundary
305 between $P = 9$ and $P = 10$ in the nested cohorts. This result separates computational feasibility from statistical scope: adding samples can improve the information available to a model, but increasing design width also raises the cost of every per-gene solve and the size of resident intermediates. Chunking, mixed precision where statistically acceptable, or a wider fused kernel may move this boundary, but each would require renewed parity analysis.

310 The practical value is greatest when the same reference analysis is applied repeatedly, including perturbation screens, resampling studies, and evaluation of generated expression profiles. cuDESeq2 is therefore complementary to methods that change the inferential model for scalability: it provides a path for retaining DESeq2 semantics when compatibility with established analyses is the priority. For isolated small experiments, GPU availability, transfer
315 costs, and software complexity may outweigh the wall-time benefit.

Several limitations bound the present conclusions. Measurements use one A100 and one 12-vCPU host, so absolute time, speedup, energy use, and cost cannot be generalized to other hardware. Five of the six primary designs contain only seven or eight samples, and the larger scaling study derives from a single GTEx source rather than independent

cohorts. Interactions, observation weights, the `ashr` and normal shrinkage estimators, and the `glmGamPoi` dispersion backend remain outside the validated scope. The fused Triton path supports $P \in \{2, \dots, 6\}$; wider designs use eager dispersion. Finally, the large-GTEx CPU endpoints and memory-boundary points are single observations, and the tissue series changes sample count and design width together.

In conclusion, gene-batched GPU execution can substantially accelerate reference-compatible negative-binomial differential expression. The findings support the use of DESeq2-style inference in high-throughput settings where computational cost might otherwise motivate a less comparable surrogate.

Funding

This work was funded by Synthesize Bio.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

The `pasilla` benchmark uses the *Drosophila melanogaster* RNA-seq experiment distributed in the Bioconductor `pasilla` 1.40.0 source package (Brooks et al., 2011). The `airway` benchmark uses the human airway smooth-muscle-cell experiment in the Bioconductor `airway` 1.32.0 source package (Himes et al., 2014). The GTEx whole-blood versus skeletal-muscle benchmark uses `recount2` gene-level data for SRA study SRP012682 (<https://recount-ompendata.s3.amazonaws.com/recount2/v2/SRP012682/>)

`rse_gene.Rdata`) (GTEx Consortium, 2020; Collado-Torres et al., 2017).

Source code, deterministic data-preparation procedures, source and prepared file check-
sums, benchmark records, and the digest-pinned software environment are archived at
345 https://github.com/synthesizebio/gpu_deseq under the MIT license. The archive
includes the exact GTEx source-column selections, raw timing observations, memory and
OOM records, endpoint timings, parity metrics, and an executable reproducibility guide.
The 1.37-GB recount2 source object is retrieved from the public URL above and is not
redistributed in the repository. The archived provenance record contains source URLs, byte
350 lengths, checksums, software versions, and cohort identifiers for the reported analyses.

References

A. K. Adduri, D. Gautam, B. Bevilacqua, et al. Predicting cellular responses to perturbation
across diverse contexts with State. *bioRxiv*, 2025. doi: 10.1101/2025.06.26.661135.

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma-poisson
355 generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702,
2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine
Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory
map between *Drosophila* and mammals. *Genome Research*, 21(2):193–202, 2011. doi:
360 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with
artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi:
10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq

- analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.
- Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.
- 370 GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.
- Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014.
- 375 doi: 10.1371/journal.pone.0099625.
- Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.
- 380 G. Koytiger, A. M. Walsh, V. Marar, et al. Generative genomics accurately predicts future experimental results. *bioRxiv*, 2025. doi: 10.1101/2025.09.08.674753.
- Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.
- 385 Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9):btad547, 2023. doi: 10.1093/bioinformatics/btad547.

NVIDIA Corporation. CUDA C++ programming guide: CUDA graphs. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>, 2024. Accessed July 2026.

390 Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

395 Christina V. Theodoris, Ling Xiao, Anant Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.
400

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for
405 sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary benchmark details

The benchmark host was a KVM virtual machine with an Intel Xeon CPU at 2.20 GHz, 6 physical cores and 12 logical CPUs, 83 GiB RAM, and Linux 5.10. GPU measurements
410 used one NVIDIA A100-SXM4-40GB. The software environment was PyTorch 2.7.1 with CUDA 12.6 and NVIDIA driver 580.126.20; the R environment was R 4.6.0 with DESeq2 1.52.0 and apeglm 1.34.0. OMP, OpenBLAS, and MKL thread counts were pinned to one for the R measurements. Main-suite and real-GTEx GPU timings are medians of five measured repetitions following an untimed warm-up. The two practical 12-worker and
415 two controlled serial direct large-GTEx R endpoints, two serial staged parity references, and four OOM-boundary probes are explicitly single-run measurements. The separate synthetic sample-axis sweep uses seven measured repetitions after three warm-ups and records its software stack in its artifact.

Supplementary algorithms

420 Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 FUSED TRITON DISPERSION FIT (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.

Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

- 1: **Launch** one Triton program for every gene g in parallel.
 - 2: Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
 - 3: Initialize $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
 - 4: **repeat** inside the same fused program
 - 5: Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
 - 6: If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
 - 7: Compute the gradient $q \leftarrow \partial \ell / \partial a$.
 - 8: Propose the gradient-ascent step $\tilde{a} \leftarrow a + \kappa q$.
 - 9: Accept or reject $\tilde{a}$ using the Armijo condition; adapt κ .
 - 10: **until** converged or *maxit* is reached.
 - 11: Store the final a_g and iteration count c_g once in global memory.
-

Supplementary Algorithm S2 DETAILED FUSED TRITON DISPERSION KERNEL (CR-MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$, $P \in \{2, \dots, 6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^G$; compile-time $h \equiv \text{HASPRIOR}$; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0 .

Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

```

1: parallel for Triton program  $g \in \{0, \dots, G-1\}$  do (one program per gene)
2:   Load  $\mathbf{y}_g$ ,  $\boldsymbol{\mu}_g$ , and  $\mathbf{X}$  once into program-local state.
3:   Set  $a \leftarrow \text{clip}(\log \alpha_g^{(0)}, -30, 10)$ ,  $\kappa \leftarrow \kappa_0$ ,  $n_{\text{acc}} \leftarrow 0$ , and  $c \leftarrow 0$ .
4:   Set  $(\ell, q) \leftarrow \text{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
5:   while  $c < \text{maxit}$  and not converged do
6:      $c \leftarrow c + 1$  and propose  $\tilde{a} \leftarrow a + \kappa q$ .
7:     If  $q \neq 0$  and  $\tilde{a} \notin [-30, 10]$ , shorten  $\kappa$  so that  $\tilde{a}$  lies on the violated boundary.
8:     Set  $(\tilde{\ell}, -) \leftarrow \text{ObjectiveGradient}(\tilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ .
9:     Set  $r \leftarrow \lceil \tilde{\ell} \geq \ell + \varepsilon \kappa q^2 \rceil$ . (Armijo acceptance flag)
10:    Select  $a' \leftarrow r ? \tilde{a} : a$ ,  $\ell' \leftarrow r ? \tilde{\ell} : \ell$ , and  $\Delta \leftarrow \ell' - \ell$ .
11:    Set  $(-, q') \leftarrow \text{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$  and  $q \leftarrow r ? q' : q$ .
12:    Set  $n'_{\text{acc}} \leftarrow n_{\text{acc}} + r$  and  $\kappa_{\text{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$ .
13:    If  $r$  and  $n'_{\text{acc}} \bmod 5 = 0$ , set  $\kappa_{\text{acc}} \leftarrow \kappa_{\text{acc}}/2$ .
14:    Set  $\kappa \leftarrow r ? \kappa_{\text{acc}} : \kappa/2$ ,  $n_{\text{acc}} \leftarrow n'_{\text{acc}}$ ,  $a \leftarrow a'$ , and  $\ell \leftarrow \ell'$ .
15:    Converge if  $r \wedge (\Delta < 10^{-6} \vee a < \log(\text{minDisp}/10))$ .
16:  end while
17:  Store  $a_g \leftarrow a$  and  $c_g \leftarrow c$  in global memory.
18: end parallel for
19: function ObjectiveGradient( $a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2$ )
20:   Set  $\alpha \leftarrow e^a$ ,  $\mathbf{W} \leftarrow \text{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$ , and  $\mathbf{B} \leftarrow \mathbf{X}^\top \mathbf{W} \mathbf{X}$ .
21:   Compute  $\ell \leftarrow \ell_{\text{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2} \log \det(\mathbf{B})$ .
22:   Compute  $q \leftarrow \partial \ell / \partial a$ , including  $\text{tr}(\mathbf{B}^{-1} \partial \mathbf{B} / \partial \alpha)$  via an unrolled  $2 \times 2$  formula ( $P=2$ ) or  $P \times P$  LU solve.
23:   if  $h = 1$  then  $\ell \leftarrow \ell - (a - m)^2 / (2\sigma^2)$  and  $q \leftarrow q - (a - m) / \sigma^2$ . (MAP only)
24:   return  $(\ell, q)$ .
25: end function

```
